

Foundations of Distributed Systems

Trustless consensus (Bitcoin), Conflict-free Replicated Data Types (CRDT), Cloud and Scaling

Chapter 3.5 / 2024-10-11

Prof. Dr. Christian Tschudin

Dept of Mathematics and Computer Science, University of Basel

Overview

- Trustless consensus: Bitcoin
- Conflict-free Replicated Data Types (CRTD)
- A gentle introduction to the cloud:
 - cloud mindset
 - Google Pub/Sub example, and (high) latency
 - scaling up and out, loadbalancing and autoscaling, sharding
 - microservices and serverless

Bitcoin (Satoshi Nakamoto, March 2009)

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network.

In this paper, we propose a solution to the double-spending problem using a peer-to-peer distributed timestamp server to generate computational proof of the chronological order of transactions. The system is secure as long as honest nodes collectively control more CPU power than any cooperating group of attacker nodes.

The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

→ a case of “**trustless computing**”

Trustless Computing

trustless = outcome does not depend on well-behavedness of peers

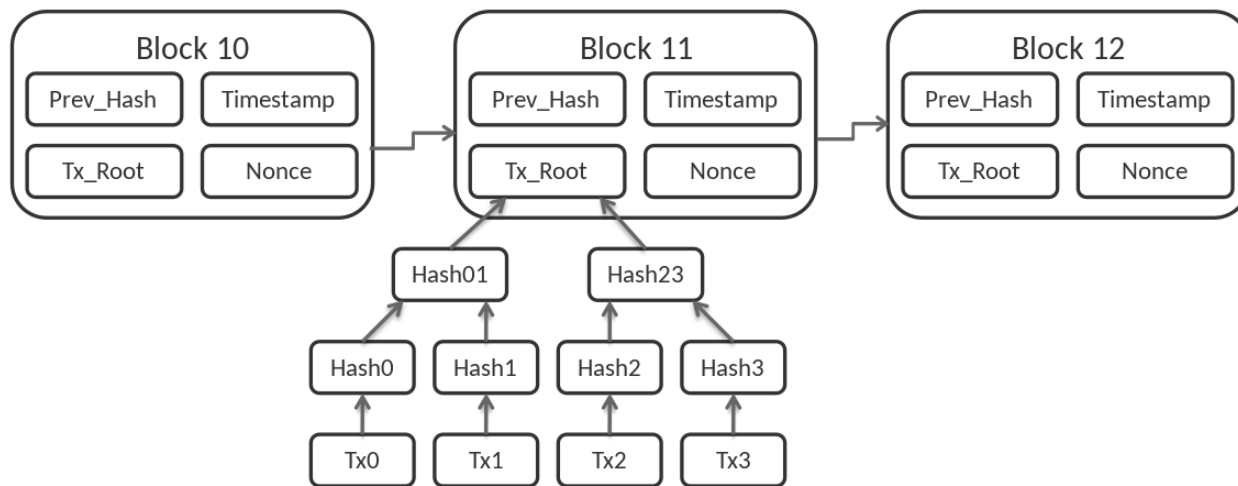
Examples:

- DNS is **not** trustless: you fully depend on operators to return valid name/ip mappings
- The Public Key Infrastructure (PKI), used for Web TLS certificates, is **not** trustless, as you fully depend on the correct behavior of Certificate Authorities to handle their root keys
- “self-sovereign identity” e.g., the public part of a self-generated ed25519 key for which only you possess the private key, **is** trustless

Bitcoin – trustless Ledger in a public Blockchain

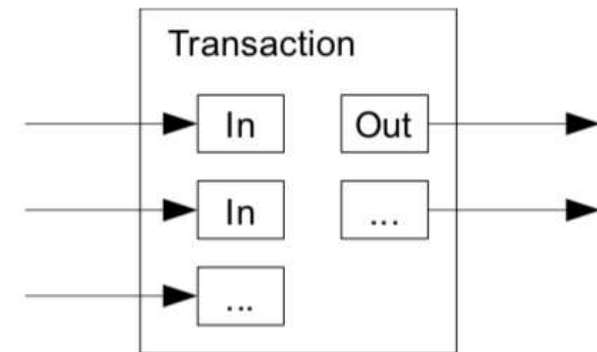
Ledger = immutable record of (financial) transactions.

Group transactions in blocks, timestamp and chain them:



Note the “nonce” field, important!

Constrain on a transaction:
sum of withdrawals (“in”) must
match sum out deposits (“out”)



Bitcoin Consensus – breaking symmetry, in time

Goal: global agreement about well-formed (Bitcoin) hash chain.

How to compute consensus, without trusting the participants?

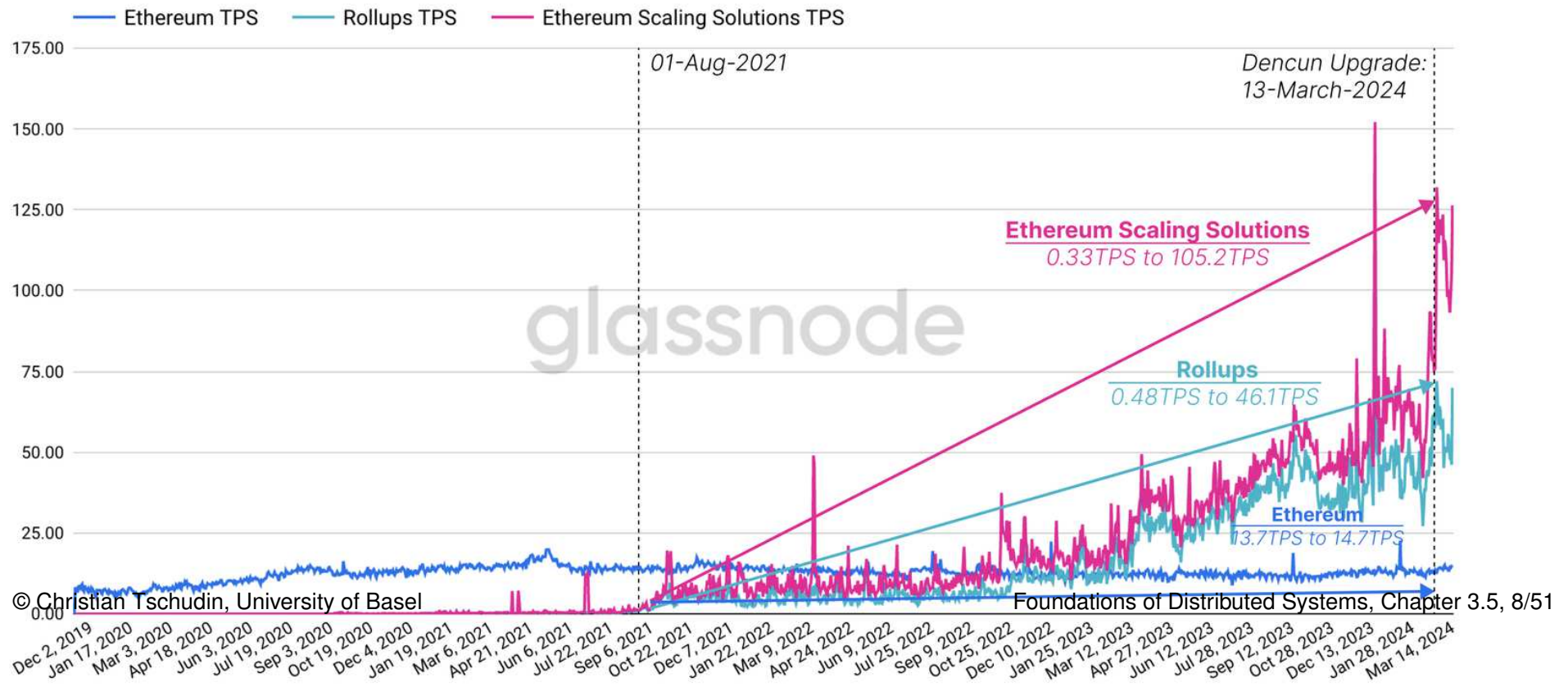
- Remember Leader Election: pick node with highest ID
 - will not work here (everybody claiming `id=0xffff...`)
 - Bitcoin: *random* selection ... **along time axis**:
 - “Hashcash puzzle”, 1997 (aka “proof of work”)
 - competitors must find appropriate nonce value
 - such that the block’s hash val has consecutive span of zeros
- `hash(block) = 000...000 110101..11`
- **first** node to find a good nonce will be the winner
 - all others must then validate, will then accept new block

Blockchain Consensus: a Scaling Problem

Throughput not glorious:

- Bitcoin has 5 to 10 transactions per sec, globally !
- BTC and Ethereum explore alternatives, still poor:

Ethereum Scaling Solutions: Average TPS



Blockchain TPS

Cryptocurrency	Transactions Per Second (TPS)	Average Transaction (Block) Confirmation Time
Bitcoin	3-7	10 min
Ethereum	15-25	6 min
Solana	2,825	0.4 sec
Polkadot	1,000	4-5 sec
EOS	4,000	0.5 sec
Cosmos	10,000	2-3 min
Stellar	1,000	2-5 sec
Dogecoin	30	1 min

From 2021: <https://phemex.com/blogs/what-is-transactions-per-second-tps>

Comparison:

– credit card networks (VISA, MC) operate at 2'000–20'000 tps

Consensus is a Bottleneck

What do “tps limits” mean?

A distributed system requiring consensus cannot scale arbitrarily.

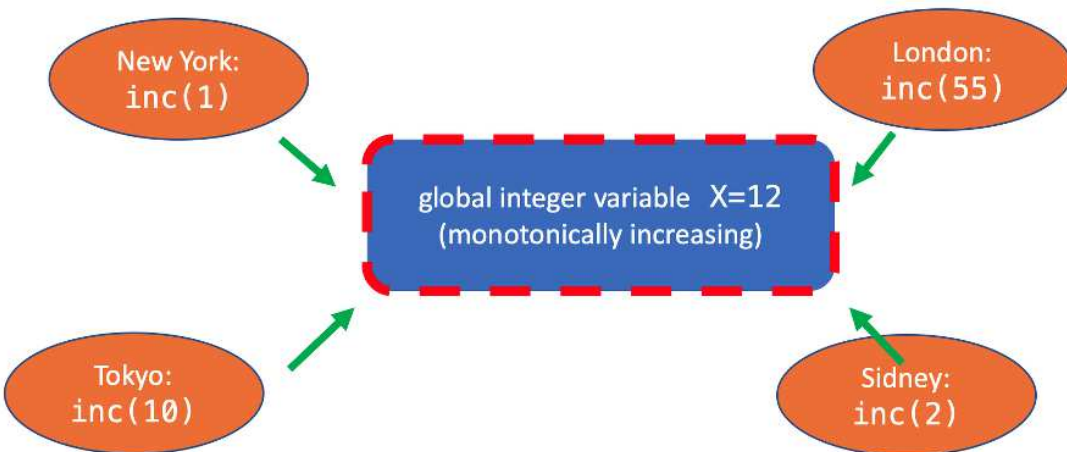
Can we do better?

Consensus so far solved by *concurrency control*:
achieves consistency by serializing the system state such that only one sequence of events is valid, globally.

→ Question: can we have consensus *without* concurrency control?

Distributed Computing ... without online consensus

Concurrency control not always needed, yet consistency
(peek preview of “Computer Networks” lecture in spring)



Example: shared global integer variable, monotonically increasing

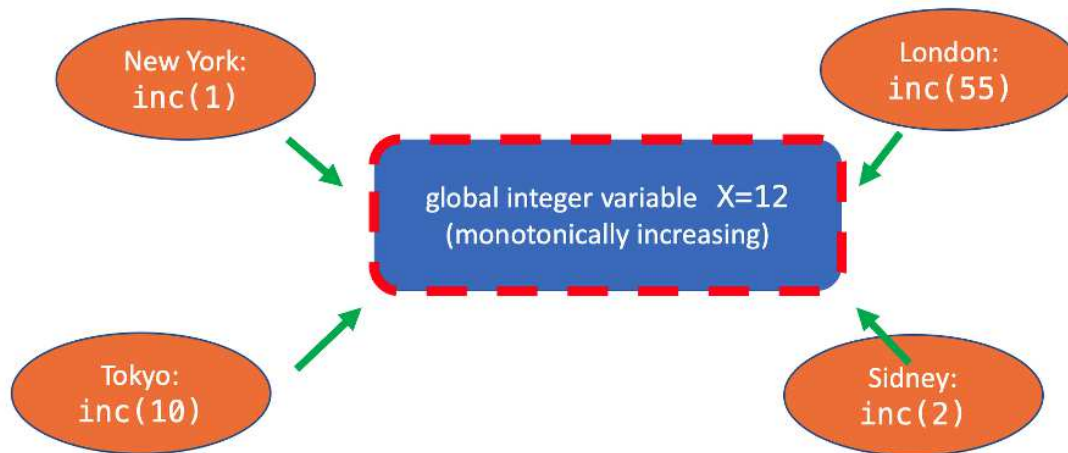
- all participants can add, in any order
- **while no coordination needed**
- yet end result will always be accurate, eventually (consensus!)

Can be achieved with a *conflict-free replicated data type (CRDT)*
("conflict-free" means "fully-scalable" because no coordination needed)

Distributed Computing ... without online consensus

We seek the following service:

- globally shared variables (of some type)
- readout: *eventual* consistency instead of “always consistent”



- one globally shared variable (int)
- implemented as replicas
- at each replica site:
 - apply local operations (inc)
- synchronization of replicas, from time to time

A use case of a shared monotonically increasing integer: “likes”

A “grow-only counter” CRDT

We want a globally shared monotonically increasing integer variable:

- implemented as a **vector with per-replica components**
- readout procedure: sum of all vector components
- use message exchange for updates (`inc` operation):
simple reliable broadcast (no ordering or atomicity constraints)

Example:

- start with e.g. $\langle 2, 3, 3, 4 \rangle$ (`val=12`)
 - assume `inc(10,0,0,0)`, `inc(0,1,0,0)`, `inc(0,0,55,0)`, `inc(0,0,0,2)`
 - can be applied in any order (commutative)
- EVENTUALLY, all replicas will be consistent** (`val=80`)

A “integer” CRDT (positive-negative counter)

Constructing a globally shared integer CRDT:

- implemented as *two* grow-only counters, I and D
 - I is used for the `inc` operations
 - D other for the `dec` operations
- readout procedure: return $readout(I) - readout(D)$

Properties:

- works even when being offline (all ops apply to local replica)
- can be temporarily inconsistent

CRDT History

Breakthrough in 2011

Shapiro, Marc; Preguiça, Nuno; Baquero, Carlos; Zawirski, Marek: “*Conflict-Free Replicated Data Types*”

In Stabilization, Safety, and Security of Distributed Systems (PDF).
Lecture Notes in Computer Science. Vol. 6976. Grenoble, France:
Springer Berlin Heidelberg. pp. 386–400.
doi:10.1007/978-3-642-24550-3_29

See also Wikipedia page

https://en.wikipedia.org/wiki/Conflict-free_replicated_data_type
and <https://crdt.tech/>

Using the Wikipedia “formalization”

Grow-only Counter

```
payload integer[n] P
  initial [0,0,...,0]
update increment()
  let g = myId()
  P[g] := P[g] + 1
query value(): integer v
  let v = sum(P[i])
compare (X, Y): boolean b
  let b = for all i in [0, n-1] : X.P[i] <= Y.P[i]
merge (X, Y): payload Z
  for all i in [0, n-1] : Z.P[i] = max(X.P[i], Y.P[i])
```

Positive-Negative Counter

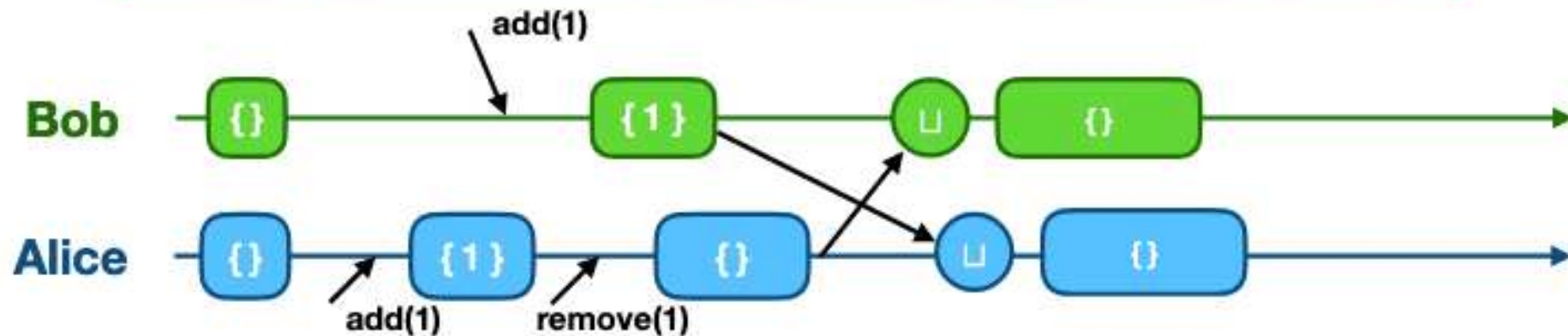
```
payload gcounter I, gcounter D
  initial I=0, D=0
update increment()
  I.increment()
update decrement()
  D.increment()
query value(): integer v
  let v = I.value() - D.value()
compare (X, Y): boolean b
  let b = forall i in [0, n-1]: X.I[i] <= Y.I[i] and
    forall i in [0, n-1]: X.D[i] <= Y.D[i]
merge (X, Y): payload Z
  forall i in [0,n-1]: Z.I[i] = max(X.I[i], Y.I[i])
  forall i in [0,n-1]: Z.D[i] = max(X.D[i], Y.D[i])
```

Grow-Only Set

```
payload set A
  initial {}
update add(element e)
  A := A union {e}
query lookup(element e): boolean b
  let b = e in A
compare (S, T): boolean b
  let b = S.A <= T.A
merge (S, T): payload U
  let U.A = S.A union T.A
```

Causal-Length Set (CLS)

We can take inspiration from Blockchains' convergence property that the *longest chain of (state-changing) operations wins*.



S_i is a dictionary (Set of key-value pairs) in which the keys, e.g. k , are the elements of the set and the values, e.g. $S_i[k]$, are the state of those elements (as Grow-Only Counters):

1. If $S_i[k]$ is odd, then k is in the set.
2. If $S_i[k]$ is even, then k is not in the set.

If a pair (k,v) does not exist in S_i , then $S_i[k] = 0$.

Causal-Length Set (CLS), contd

With CLS, elements can be added and removed arbitrarily often

```
payload dict A
```

```
    initial {}
```

```
update add(element e)
```

```
    if not e in A then A[e] = 1
```

```
    else if A[e] even then A[e] = A[e] + 1
```

```
update remove(element e)
```

```
    if e in A and A[e] odd then A[e] = A[e] + 1
```

```
query lookup(element e): boolean b
```

```
    let b = (e in A) and (A[e] odd)
```

```
merge (S, T): payload U
```

```
    for all e in (S.keys() union T.keys()): U[e] = max(S[e], T[e])
```

CRDT: Concluding Remarks

“conflict-free” as a way to have eventual consensus, without having to negotiate

- fully local operations (like Git)
- synchronization, from time to time (like Git)
- never have merge conflicts (unlike Git)

Several CRDT “styles” (state-based CRDTs, operation-based CRDTs), designing CRDTs not easy,

→ see “Computer Networks” lecture this coming spring

Overview

- Trustless consensus: Bitcoin
- Conflict-free Replicated Data Types (CRTD)
- A gentle introduction to the cloud:
 - cloud mindset
 - Google Pub/Sub example, and (high) latency
 - scaling up and out, loadbalancing and autoscaling, sharding
 - microservices and serverless

The Cloud Mindset

Evolution of machines and mindset:

- from mainframes to NOW (Network of Workstations)
- mindset: embrace failures, service-oriented architectures

One computer is nothing, recovery is everything

2001: landmark paper by Brown and Patterson “*Embracing Failure: A Case for Recovery-Oriented Computing (ROC)*”

- Assumption in BSc Computer Sci curriculum: reliable computing, storage etc. Reality: failures are inevitable, the norm
- Change your mindset:
 - a single computer instance has no value,
 - but overall **service** has (SOA=Service-oriented Applications)
- Don't rely on stability but prepare to recover, all the time

Decompose such that you can reconfigure, restart “at will”. Why?

“retirement notif.” (even cloud components break down)

Date: Wed, 14 Sep 2016 04:57:11

From: “Amazon Web Services, Inc.” <no-reply-aws@amazon.com>

To: christian.tschudin@unibas.ch

Subject: [Retirement Notification] Amazon EC2 Instance scheduled for retirement.

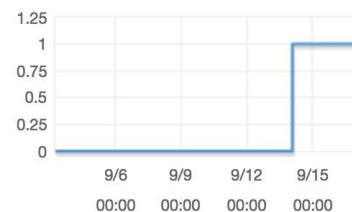
Dear Amazon EC2 Customer,

We have important news about your account (AWS Account ID: 299213182874). EC2 has detected degradation of the underlying hardware hosting your Amazon EC2 instance (instance-ID: i-1db7bd3d) in the us-east-1 region. Due to this degradation, your instance could already be unreachable. After 2016-09-28 02:00 UTC your instance, which has an EBS volume as the root device, will be stopped.

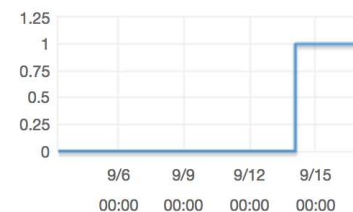
* How does this affect you?

...

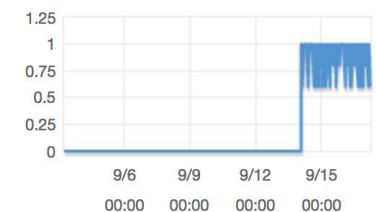
Status Check Failed (Any) (Count)



Status Check Failed (Instance) (Count)



Status Check Failed (System) (Count)



From Slide Deck of Jeff Dean, Google, 2009

The Joys of Real Hardware

Typical first year for a new cluster:

- ~0.5 **overheating** (power down most machines in <5 mins, ~1-2 days to recover)
- ~1 **PDU failure** (~500-1000 machines suddenly disappear, ~6 hours to come back)
- ~1 **rack-move** (plenty of warning, ~500-1000 machines powered down, ~6 hours)
- ~1 **network rewiring** (rolling ~5% of machines down over 2-day span)
- ~20 **rack failures** (40-80 machines instantly disappear, 1-6 hours to get back)
- ~5 **racks go wonky** (40-80 machines see 50% packetloss)
- ~8 **network maintenances** (4 might cause ~30-minute random connectivity losses)
- ~12 **router reloads** (takes out DNS and external vips for a couple minutes)
- ~3 **router failures** (have to immediately pull traffic for an hour)
- ~dozens of minor **30-second blips for dns**
- ~1000 **individual machine failures**
- ~thousands of **hard drive failures**

slow disks, bad memory, misconfigured machines, flaky machines, etc.

Failure Rates, High Availability and Durability

Contrast above failure rates with Amazon's service characteristics of its Simple Storage Service (S3):

- 99.9999999999% durability
average annual expected loss of 0.0000000001% of objects:
“if you store 10,000 objects with Amazon S3, you can on average expect to incur a loss of a single object once every 10,000,000 years”
- For comparison: 99.99% availability means
→ **unavailable** < 8.6sec per day see <https://uptime.is/>

Scaling – Starting from Limits

Numbers Everyone Should Know

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	100 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	10,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from network	10,000,000 ns
Read 1 MB sequentially from disk	30,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

Scaling – 1 Mio DB writes/s – how?

Cassandra Hits One Million Writes Per Second on Google Compute Engine

Thursday, March 20, 2014

Google is [known for creating scalable high performance systems](#). In a recent blog post, we demonstrated how Google Cloud Platform can rapidly provision and scale networking load to handle [one million requests per second](#). A fast front end without a fast backend has limited use, so we decided to demonstrate a backend serving infrastructure that could handle the same load. We looked at popular open source building blocks for cloud applications and choose [Cassandra](#), a NoSQL database designed for scale and simplicity.

Using 330 [Google Compute Engine](#) virtual machines, 300 1TB Persistent Disk volumes, Debian Linux, and Datastax Cassandra 2.2, we were able to construct a setup that can:

- sustain one million writes per second to Cassandra with a median latency of 10.3 ms and 95% completing under 23 ms

Scaling, three scale-out strategies

Scaling **Up and Out** (vertical and horizontal)

- up == buy/choose better hardware
- out == add more instances ← the cloud approach

A) **Scale out: (dynamic) loadbalancing** among replicas

- spread the load, at many levels

DNS, front web tier, DB instances

B) **Scale out: federating**/subdividing services

- specialized sub-DBs: forum, product, internal etc

C) **Scale out: (fixed) sharding** – hash values as names
deterministically spread DB content to set of DB instances

Q: How to achieve high Durability?

A: Redundancy . . . and more: from Amazon's service description

- *“S3 is designed to sustain the concurrent loss of data in two facilities”*
→ **three-fold** replication (two storage sites can go down)
- *“a functioning, regularly tested backup.”*
- *“quickly detecting and repairing any lost redundancy”*
- *“S3 also regularly verifies the integrity of your data using checksums”*

THIS is the service, really.

Cloud Economics

CapEx = (capital) expenses: buying server, building data center

OpEx = (operational) expenses: like electricity, humans

- Cloud operator takes over all CapEx and sells services to you at OpEx + margin
- Great for **startups**:
buy compute services on demand, without CapEx

OpEx model has also disadvantages: less stable, less predictable while other dimensions similar, e.g. level of required expertise

Some Cloud Terminology

HPC vs Cloud (and the older “Grid”)

Of Grid, Clouds and HPC

Grids

- Pioneered the *service* model.
- Latency intrinsic to distributed systems.
- Less suitable for differential equations.
- Dependent on remotely installed software stack.

Clouds

- Elasticity on demand.
- Poor locality (affinity) affects sci. comp.
- Even less suitable for differential equations.
- Ship all your software with your image.
- MapReduce and GFS.

HPC

- Best suited for differential equations and tight MPI.

<https://scitech.isi.edu/presentations/2011/grits-caltech-talk.pdf>

Peter Du's History of the Cloud

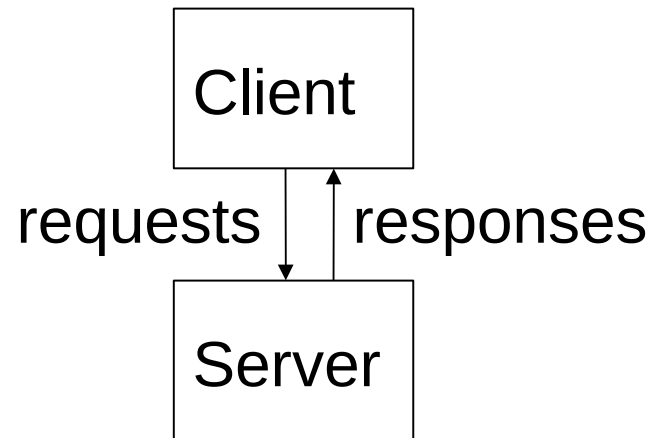
see external slide deck

Distributed Systems

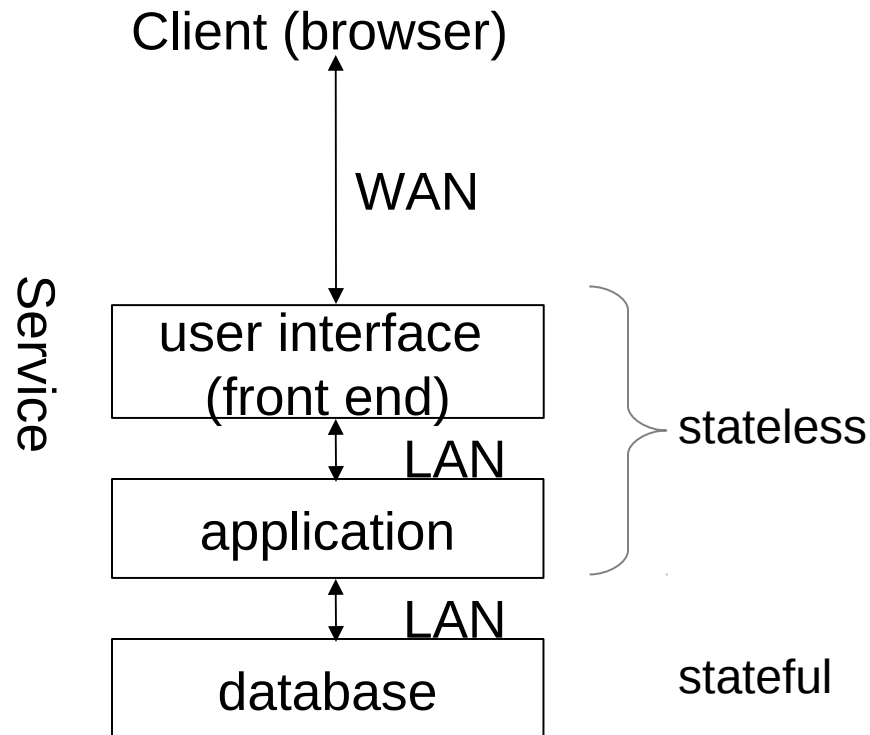
[Fall 2013]

Lec 3: Example use cases (continued)
Cloud computing

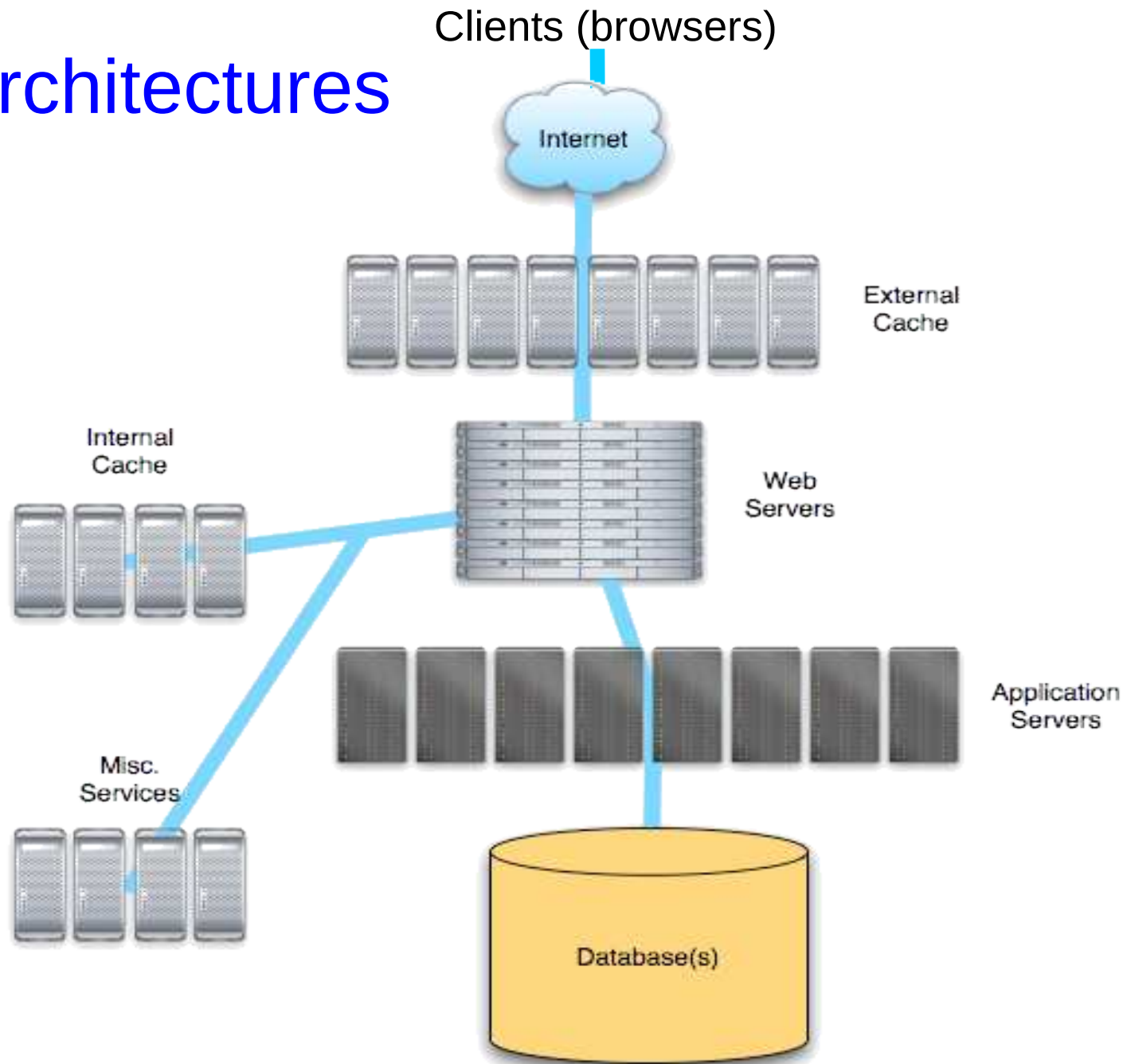
1. The Client-Server Model



2. The Three-Tiered Architecture



3. Real Architectures



What is cloud computing?

- Computing technology in which data and/or computation are outsourced to a massive-scale, multi-user infrastructure that is managed by a third-party.
- Appeared gradually due to two important challenges facing the Web:
 - Scaling
 - Management
- I'll tell you the story of how it appeared, so as to help you understand what it is

Around 1995: The Scaling Challenge

- The Web and e-commerce were gaining traction



- Their challenge: [how to scale](#)?
 - 1996 to 1997: eBay grew from 41,000 to 341,000 users!

Pre-1995 Answer: Big, Expensive Computers

- Example: eBay used Sun E-10000 “supermini”
 - 64 processors @250MHz, 64GB RAM, 20TB disk, ~\$1M
- The good:
 - Easy to manage
 - Easy to program
 - Simple failure mode
- The bad:
 - Q: Any ideas?



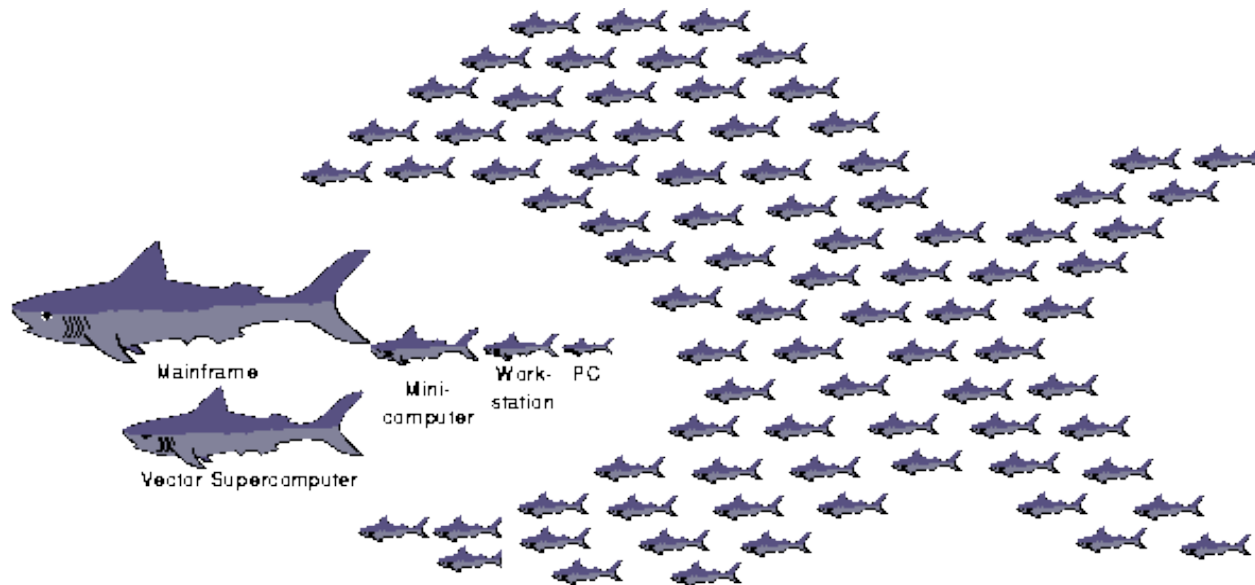
Pre-1995 Answer: Big, Expensive Computers

- Example: eBay used Sun E-10000 “supermini”
 - 64 processors @250MHz, 64GB RAM, 20TB disk, ~\$1M
- The good:
 - Easy to manage
 - Easy to program
 - Simple failure mode
- The bad:
 - Expensive
 - Single point of failure
 - No incremental scalability



1995: Berkeley Network of Workstations (NOW)

- Idea: Leverage many interconnected small, cheap, general-purpose machines for incremental scalability and reliability
 - Typical PC: 200 MHz CPU, 32MB RAM, 4GB disk



NOW-0

- 1994: NOW had 4 HP-735's



NOW-1

- 1995: NOW had 32 Sun SPARC stations



NOW-2

- 1997: 60 Sun SPARC-2's
- Build Inktomi app



Companies Adopt NOW

- Everybody builds their own clusters and grows them to handle more and more load
 - Examples: eBay, Amazon, Google, all .com bubble companies
- Similar to early days of electricity when everyone built their own generator

Q: What do you think happened next?

Late 1990s: The Manageability Challenge

- Hard to manage and program large clusters
 - How to write scalable distributed programs?
 - How to debug large-scale programs?
 - How to make services reliable?
 - How to architect the network infrastructure?
 - How to provision a cluster to handle peak load?
 - How to administer a huge number of computers?
 - ...
- Each company had to build own complex software
 - Like each of us building an OS from scratch!

Early 2000s: Scalable Cluster Primitives

- Very few technically strong companies create powerful scalable and reliable **primitives for cluster management and programming**
- Examples:
 - Google's Map/Reduce
 - The Google File System (GFS)
 - Google's Bigtable
 - Amazon's Dynamo
 - Distributed debugging and tracing tools
 - Datacenter temperature regulators
 - Scalable distributed communication mechanisms
 - ...

Mid 2000s: Three Valuable Commodities

- Giant-scale clusters with enormous excess capacity
 - Everybody provisioned for peak

Q: How big was a typical Google datacenter around 2005?

- a. 1,000 machines
- b. 5,000 machines
- c. 10,000 machines
- d. 50,000 machines
- e. 100,000 machines

Mid 2000s: Three Valuable Commodities

- Giant-scale clusters with enormous **excess capacity**
 - Everybody provisioned for **peak**
- **Expertise** for managing and operating clusters **at low cost**
 - “Economies of scale”
- **Complex software** to help program/manage clusters
 - Even full applications (e.g., Gmail, Google Calendar, etc.)

Q: What do you think happened next?

2006: Cloud computing

- AWS sells resources, expertise, and access to cloud primitives in a [pay-for-what-you-use](#) model
 - Resources: CPU, network bandwidth, persistent storage
 - Cloud primitives: Amazon S3, EC2, SQS, Map/Reduce, ...
- Google launches [Google Apps for Your Domain](#)
 - Customizable Gmail, Google Docs, Google Calendar under a custom domain (e.g., `gmail.cs.columbia.edu`)
- Google then launches the [App Engine](#)
 - Web hosting infrastructure (such infrastructures existed before, but didn't come with many primitives)
- Microsoft launches [Azure](#) in 2009

Advantages of cloud computing

- Low barrier of market entry for startups
- Cheaper, low-management email, calendars, CRM solutions
- New mobile applications
- Faster batch processing via parallelization across many machines

What do Clouds have to do w/ distributed systems?

- Clouds are powered by (and sell) distributed, scalable systems, which can be used as building blocks for easy bootstrap of new applications and services
 - Often times, you hear about clouds as being great because you don't have to purchase machines upfront
 - I think their major advantage lies in fact in the scalable services they provide
- This is unlike prior Web hosting infrastructures that predated “clouds” by many years
 - Those offered (and some still do) bare-metal and no add-on value-added services

What is the Cloud? Beyond Data Centers: XaaS

The central and magic word: Service!

What is “The Cloud”

Cloud computing refers to logical computational resources accessible via a computer network.

SaaS: Software As A Service
(e.g. Google docs)

PaaS: Platform As A Service
(IaaS + core software; e.g. AppEngine)

IaaS: Infrastructure As A Service
(e.g. Get Cycles with Web API for Cash)

Clusters are still a crucial concept.

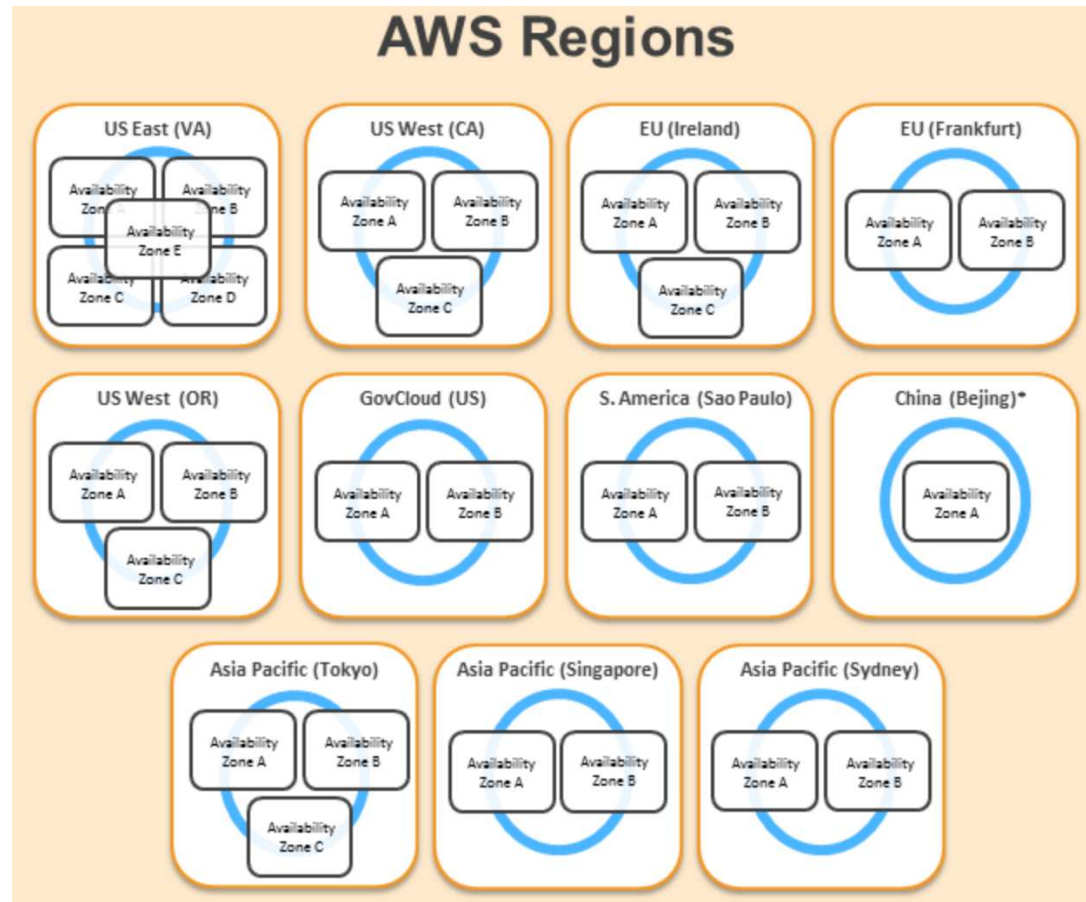
RAFT and “Availability Zones” (contd)

RAFT alone does not guarantee High Availability (HA)

- If all $3f + 1$ ($2f + 1$ with Paxos) nodes are in one computing center, a single disaster can ground your service.
- Plausible natural disasters or system failures:
 - flooding, fire
 - power outage
 - cut fiber in front of the door
 - misconfiguration of the network fabric
 - Murphy is everywhere
- The cloud to the rescue ... ?

“Availability Zones” in the Cloud

Cloud clients can choose *where* their virtual machines should run.



“Regions” make sense,
but what is a
“availability zone?”

A Region has multiple Availability Zones

Availability Zone = independent data center (DC) with

- separate power and Internet connectivity
- low latency network connection among AZs in the same region
- a region's AZs are co-located (5-15 miles apart)
such that these AZs *feel as if* belonging to one DC

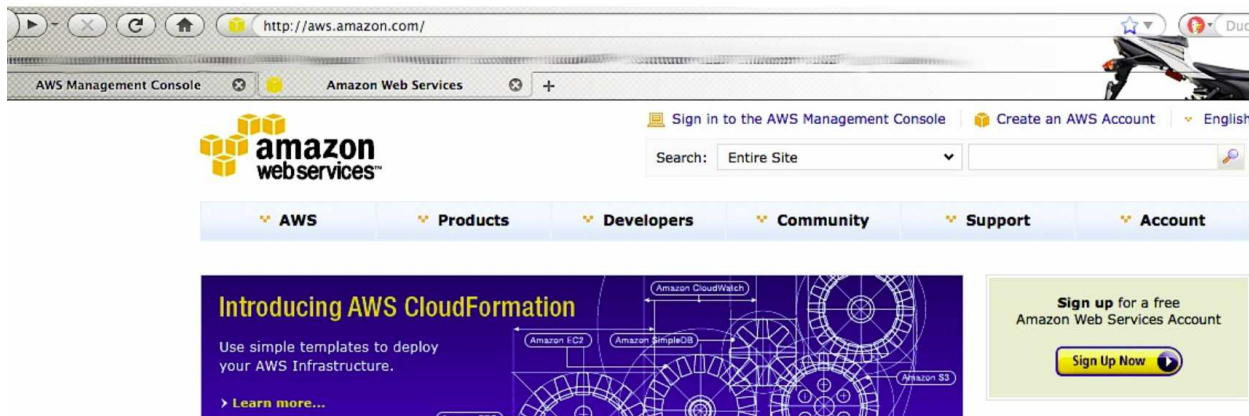
How to turn a “raft-ified app” into a high-availability service:

- Place each RAFT node in a different AZ
- but do not spread a RAFT cluster across regions
(because of delay due to network round-trip time)

How to start

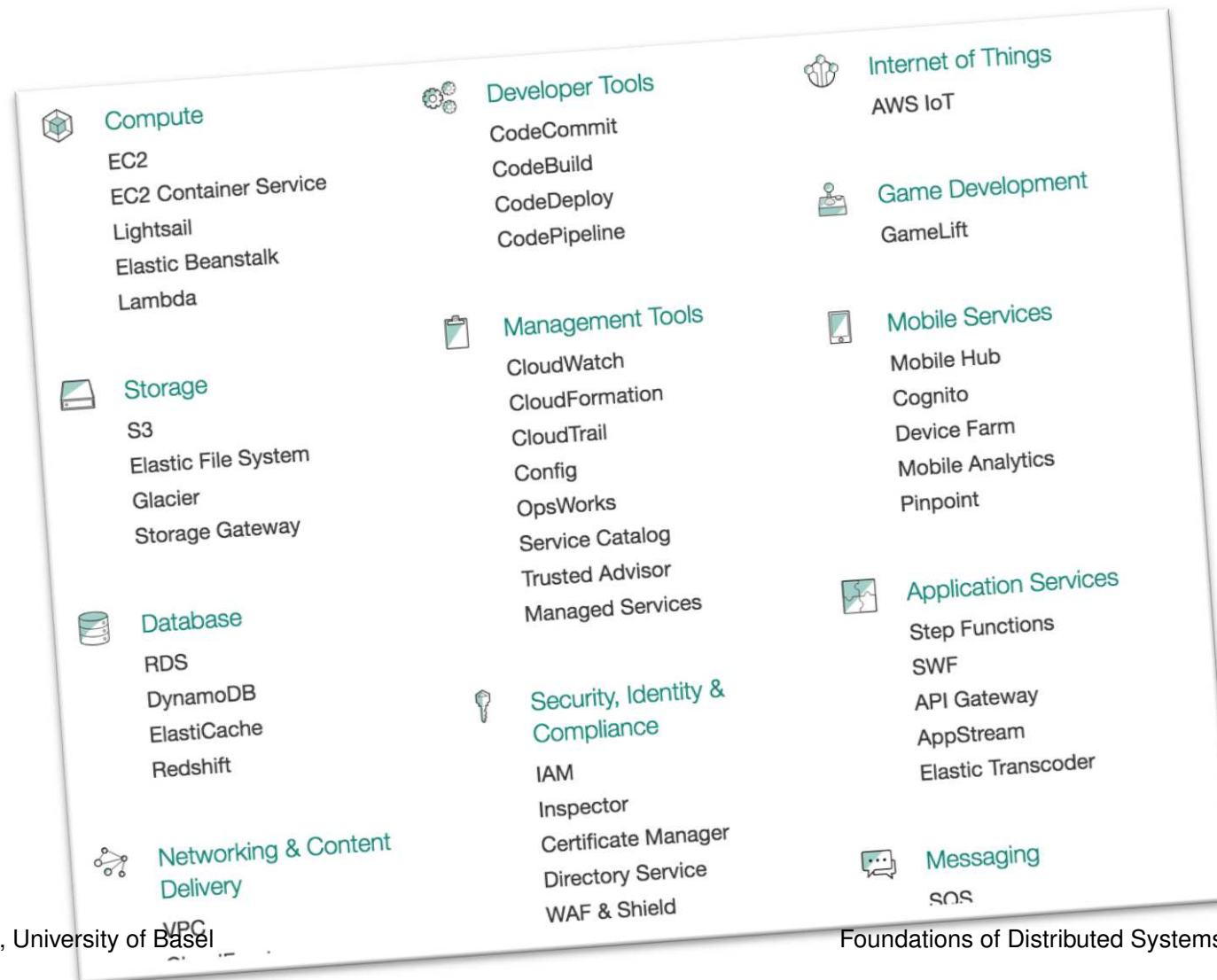
How Do I Get Started

- 1) You need a web browser
- 2) And a credit card



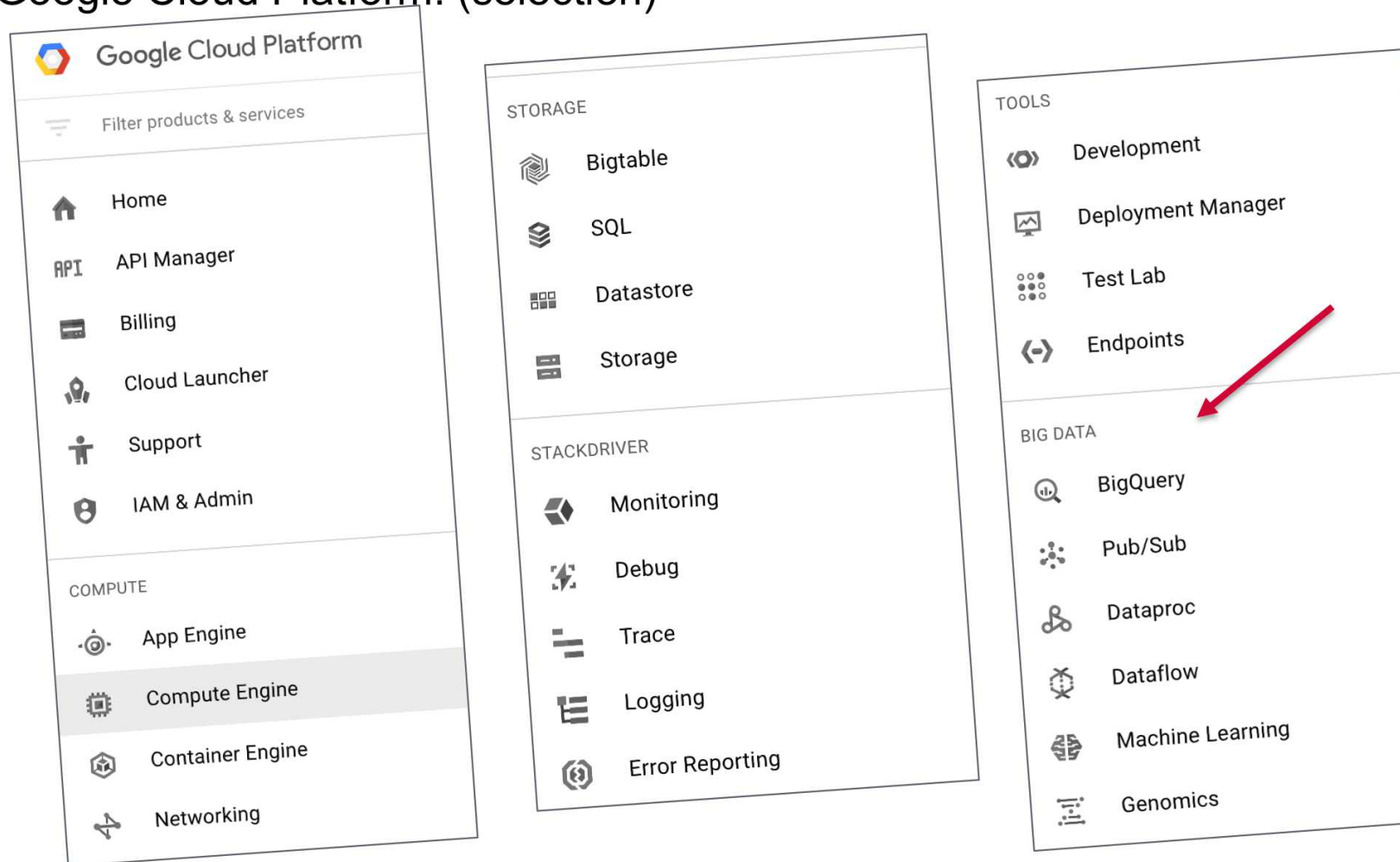
Explosion of cloud services since 2006 .

- Amazon Web Services



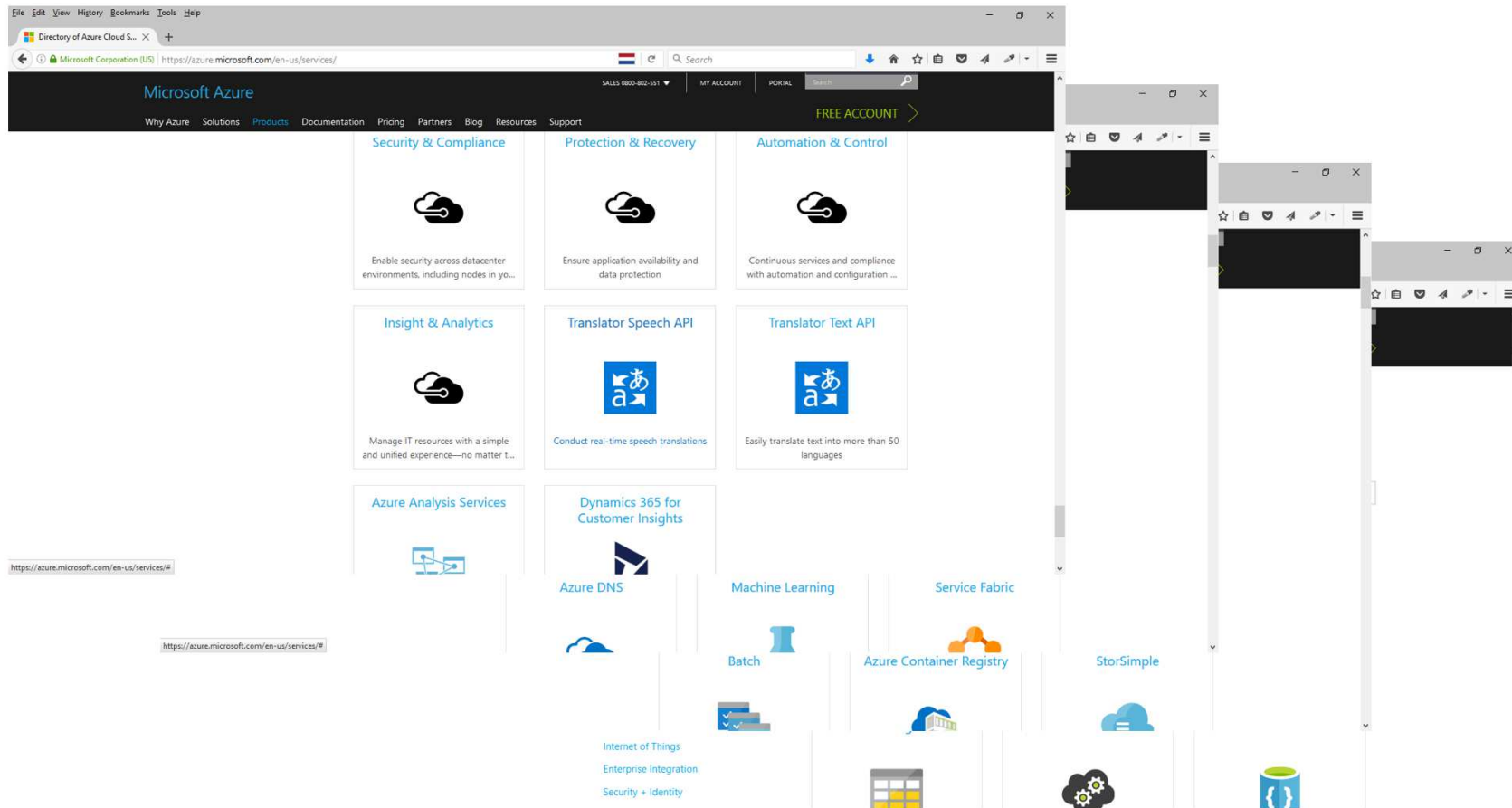
Explosion of cloud services since 2006 ..

- Google Cloud Platform: (selection)



Explosion of cloud services since 2006 ...

- Example: Microsoft Azure (<https://azure.microsoft.com/en-us/services/>)



How to start: Become a Cloud Mechanic

- public cloud, virtual private cloud (VPC), ...
- EC2, EBS, S3, SNS, SQS ...
- loadbalancer (ELB), elastic IP, security groups, route 53 ...
- app engine, pub/sub, AWS Lambda ...

Quite some concepts to learn. We pick one: message queues, e.g.

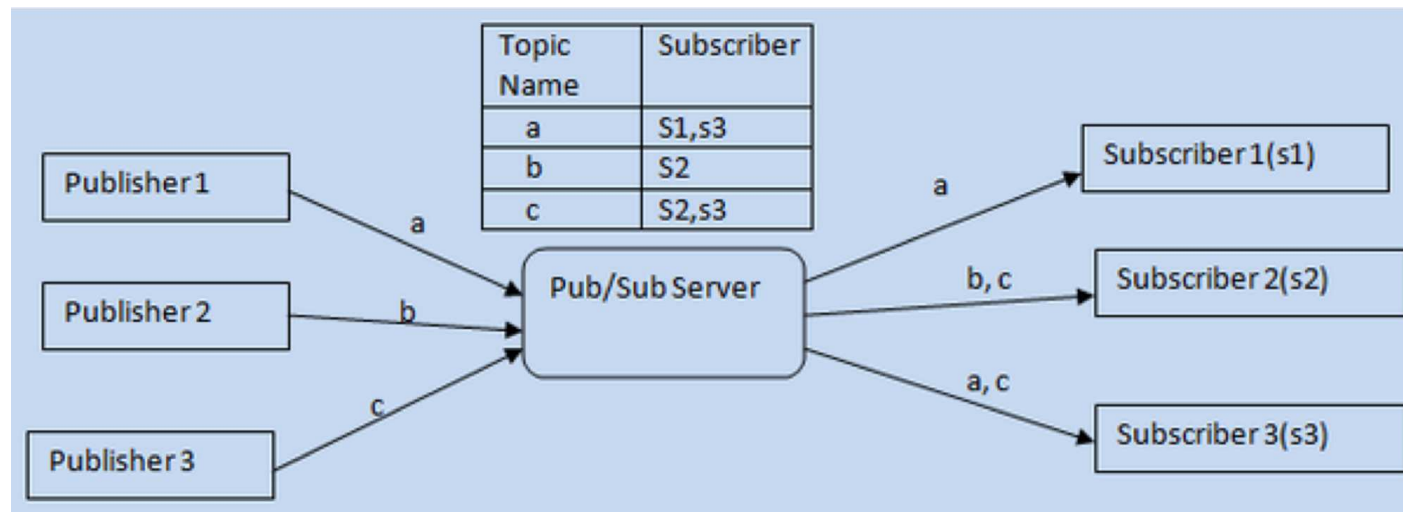


Google pub/sub

(used, among other, for cool concepts like “serverless computing”)

What is Pub/Sub?

Proposed around 2000 as a Wide Area Network technology
(Antonio Carzaniga, now prof at USI, Lugano)



Decoupling of senders and receivers (and their addresses):

- send to a *topic* (aka “channel”)
- subscribe to a *topic* and get notifications

Google Pub/Sub – reliable, persistent message queues

- Goal 1: want to have (**reliable**) “mailbox abstraction”

History: exclude shared memory, use a buffer to couple distributed processes

See the “Actor model” (Gul Agha, **1986**)

Locally in UNIX: FIFO queues for IPC (inter-process communication)

- Goal 2: make the message queue **persistent**

*“Google Cloud Pub/Sub brings the **scalability**, flexibility, and **reliability** of enterprise message-oriented middleware to the cloud. By providing many-to-many, asynchronous messaging that decouples senders and receivers, it allows for secure and **highly available** communication between independently written applications. Google Cloud Pub/Sub delivers low-latency, **durable** messaging that helps developers quickly integrate systems hosted on the Google Cloud Platform and externally.”*

How low is Google Pub/Sub's “low-latency”?

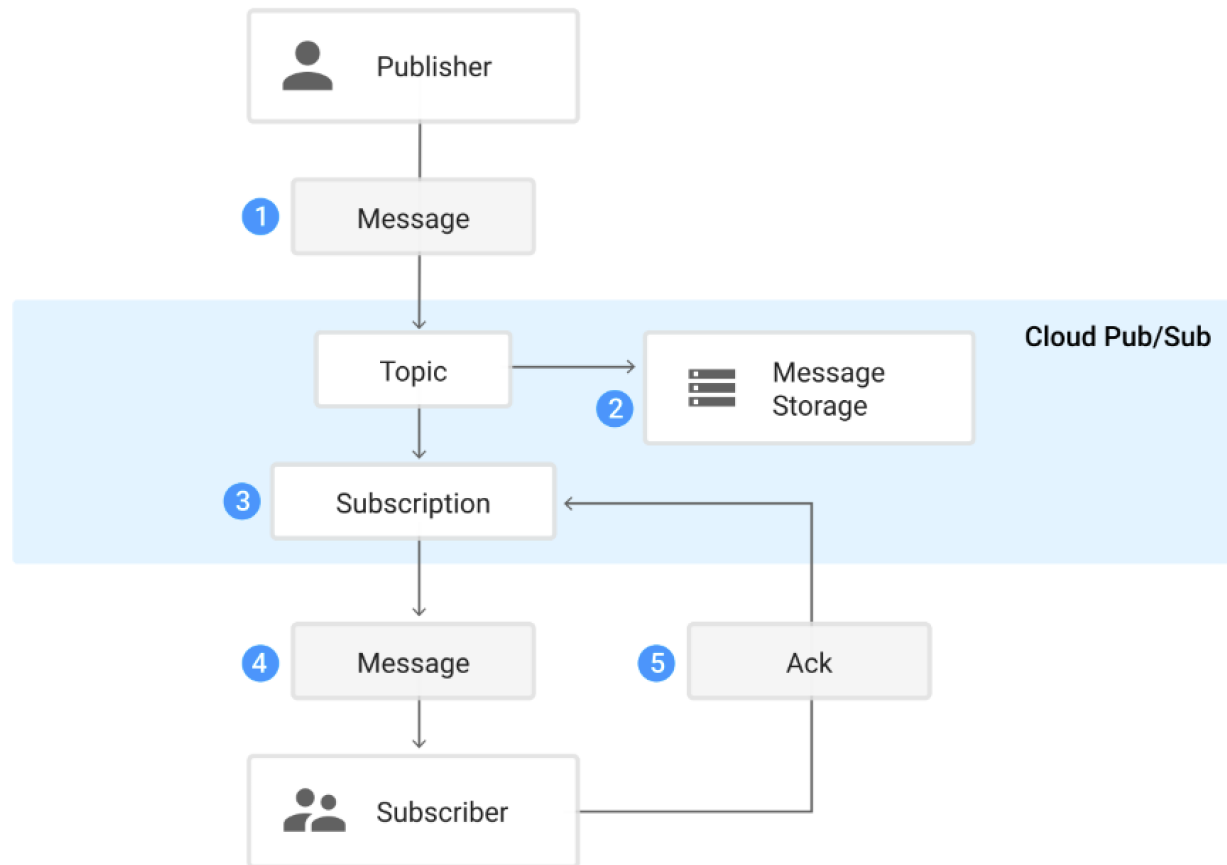
Despite Google's “low-latency” claim, numbers are disappointing:

Sep 2016: *“With Google's gcloud python library, we are seeing avg latencies around 400ms publishing tiny messages. This is unacceptable so we tried the experimental RPC library in both python and go, and see average latencies around 120ms.*

Avg 120ms latency still means: 8 messages per seconds **or less!**

(If application supports this: use streaming mode instead of per-message request-reply pattern.)

Structure of Google Pub/Sub



Need to **reliably persist** each incoming message (up to a week).

Implication:
must have some form of
RAFT-style log replication,
which explains latency

Note that delivery at subscriber
is “at-least-once”

Google Pub/Sub == “ordered broadcast”? No!

Read the spec again – no mentioning of order!

Ordered delivery is an *additional* service,

- sender must provide an “ordering key”,
- subscribers set a “enable ordering” flag (for a specific key)

- scaling is challenging, therefore *service limited to a region*
- even inside a region quite challenging, see discussion

<https://medium.com/google-cloud/google-cloud-pub-sub-ordered-delivery-1e4181f60bc8>

- Google spec: *rate limit* at publishing time is 1 MB/s (year 2020)

Video “Scaling up to your first 10 Mio users”

This talk re-given every year. An old one from “*AWS re:Invent 2015*”

<https://www.youtube.com/watch?v=vg5onp8TU6Q>

watch it!

Take-home message for the cloud:

1 user: use the vanilla cloud services

100 users: use the vanilla cloud services

10'000 users: use the vanilla cloud services

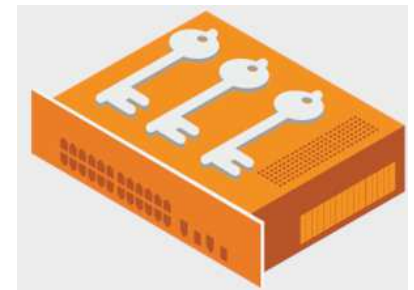
1'000'000 users: use the vanilla cloud services

10'000'000 users: perhaps start using special tricks
(sharding, federation, NoSql. . .)

Trusting the Cloud? Alternatives, Variations

1. Run your own datacenter
2. Server hotel / BYOS (bring your own server):
rent a rack in a datacenter building
Example: netric's Datacube in Münchenstein
3. Hybrid model: keep some servers “on-prem” (on-premise),
mostly done for security reasons

Also: (secure) co-processors in the cloud
typically done with “HSM”: *hardware security module*
for storing crypto keys, doing key rotation



In the Future: Trustless Computing in the Cloud

Example from EPFL: *Truly privacy-preserving federated analytics for precision medicine with multiparty homomorphic encryption*
by Fröhlicher et al (Oct 11, 2021 in *Nature Communications*)

We propose FAMHE, a novel federated analytics system that, based on multiparty homomorphic encryption (MHE), enables privacy-preserving analyses of distributed datasets by yielding highly accurate results without revealing any intermediate data.

Trick: compute on *encrypted* data (MHE)
→ forthcoming lecture of Prof Wagner